

Fucheng Zhu fuchengz@usc.edu

Abhigna Veerendra Kumar veerendr@usc.edu

Shivani Rao raoshiva@usc.edu

Sell-Through Co-Pilot

Plan re-stocking, pricing, and promotions together — fast, explainable, and built on widely used open tools.

1) Problem & why it matters

Promotions and ads bring visitors, but if items aren't available you waste money and disappoint customers; if you over-buy "to be safe," returns and leftovers burn cash. Teams often choose re-stocking rules, discounts, and ad budgets in separate silos, so campaigns run into run-outs, followed by markdowns and wasted ad spend. We propose a realistic, operational way to plan **re-stocking + discounts + ad pacing together** that runs fast enough to be used interactively.

2) Prior art we will build on (in-use, accessible)

- **Meta Robyn (open-source MMM)** — from Meta Marketing Science; provides response curves and budget/discount optimization workflows with docs and case studies you can run now. ([Facebook Experimental](#))
- **Uber Orbit (open-source forecasting)** — from Uber; daily Bayesian time-series forecasting with uncertainty, used internally for planning; GitHub + Uber blog. ([GitHub](#))
- **Odoo Inventory (open-source ERP)** — production "Reordering Rules" and **Replenishment** scheduler to propose purchase orders back to target; full user docs. ([Odoo](#))
- **ERPNext (open-source ERP)** — **Auto Reorder**: when stock drops below a configured level, it auto-creates a **Material Request** for purchase/transfer/production. ([Frappe 文档](#))
(*External validation in public health*): **OpenBoxes** "Reorder Report" and **OpenLMIS** deployments document min/reorder/max workflows and reduced stock-out rates. ([OpenBoxes 知识库](#))

3) What we will build (high level)

3.1 Ground-truth simulation layer

Use **Orbit** to forecast baseline demand (mean + uncertainty). Use **Robyn** to produce promo/ads lift curves and "what-if" scenarios for discount & spend. Feed these into a slow but accurate **inventory play-out** (lead-time noise, returns). In parallel, script **Odoo** (Reordering Rules / Replenishment) and **ERPNext** (Auto Reorder → Material Request) to capture their native re-stocking proposals under the same scenarios; log all outcomes. ([Uber](#))

3.2 Fast surrogate on GPU

Train a small tabular surrogate (e.g., **XGBoost** with `gpu_hist`) to predict: money made, on-time delivery, leftover units, and ad-waste, from product traits, delivery-time stats, **re-stocking rule**, **discount**, and **channel budgets**. XGBoost's GPU docs and treemethod references support this directly. ([XGBoost 文档](#))

3.3 Scaled search + “trust-but-verify”

Use a small Ray/Dask cluster to (a) generate many candidate plans, (b) score with the surrogate in parallel, and (c) **re-simulate only finalists** with the slow engine. Ray's scaling guides show these patterns. ([Ray](#))

4) Inputs, decisions, outputs

Inputs

Products/variants; unit cost & price; daily sales history; expected returns & time-to-return; supplier lead-time stats; promo calendar and candidate discounts; planned ad traffic by channel; practical limits (smallest order allowed, case pack, weekly cash cap, receiving capacity).

Decisions we recommend

- **Re-stocking targets/rules** per product (simple, explainable rules such as min–max or order-up-to, aligned with Odoo/ERPNext). ([Odoo](#))
- **Promo/discount level** that meets the sales goal without causing run-outs or creating large leftovers (via Robyn what-ifs). ([Facebook Experimental](#))
- **Ad pacing** (turn spend down when run-out risk is high; lean in when coverage is safe).

Outputs for leaders

- **Promo readiness** light (green / yellow / red).
- Suggested **re-stocking**, **discount**, and **ad level**.
- **Money view**: projected sales, ad spend avoided on unavailable items, leftover value avoided.
- **Customer view**: on-time delivery & risk of running out.
- **Ops view**: inbound loads vs dock capacity; weekly cash usage.

5) Methods (concrete)

Forecasting & promo effects (traditional)

Fit **Orbit** at daily cadence; back-test. Fit **Robyn** on sales/spend/discount to learn channel response & saturation; export response curves and budget splits for target windows (e.g., promo week). ([GitHub](#))

Inventory play-out (traditional)

For each joint plan, simulate day-by-day demand = Orbit baseline + Robyn lift; apply lead-time draws, a simple re-stocking rule, returns, and constraints (smallest order, case pack, cash, receiving). In parallel, run **Odoo Replenishment** and **ERPNext Auto Reorder** to capture their purchase suggestions; store all outcomes as labeled rows. ([Odoo](#))

Surrogate learning (AI model)

Train **XGBoost-GPU** to map (features + plan) → (cost, on-time rate, leftovers, ad waste). Use early stopping and, where helpful, monotonicity constraints; report feature importance for explainability. ([XGBoost 文档](#))

Scaled search (systems)

Use **Ray** to parallelize plan generation and surrogate scoring; **re-simulate only the top-K** per product group/campaign for accuracy. ([Ray](#))

6) Evaluation plan (three scenarios)

1. **Normal week** — compare baseline rules vs Co-Pilot plan.
2. **Big weekend promo** — slide discount & ad traffic; show Co-Pilot raises pre-build just enough, keeps run-out risk low, avoids leftover spike.
3. **Influencer spike** — toggle “viral risk”; Co-Pilot lifts safety buffers, splits purchase orders, trims ads/discount to survive the surge.

Reported metrics

Promo revenue ↑ with run-outs ↓; ad waste ↓ (fewer clicks to items that can’t ship); leftovers & tied-up cash ↓; planning time ↓ (surrogate vs slow sim); ablations for lead-time variance and returns.

7) Milestones (5 weeks)

- **W1–2** — Data plumbing; **Orbit** + **Robyn** on a small example; **Odoo/ERPNext** stood up for 3–5 products; first slow simulations logged. ([Uber](#))
- **W3** — Surrogate trained on GPU; validate on hold-out plans; basic dashboard (readiness light + risk timeline). ([XGBoost 文档](#))
- **W4** — Distributed search with Ray; trust-but-verify loop; influencer-spike switch. ([Ray](#))
- **W5** — Final evaluation on three scenarios; short video demo; write-up.

8) Risks & mitigations

- **Helper bias near tight constraints** → Always re-simulate finalists; add penalties for risky edges.

- **Data gaps for returns or ad forecasts** → Start with simple defaults; improve as signals arrive.
- **Seasonality drift** → Re-fit Orbit/Robyn monthly; re-train surrogate weekly on recent simulations.

9) References (accessible, being-used components)

- **Robyn** overview & docs; Meta's open MMM package. ([Facebook Experimental](#))
- **Robyn** GitHub repo. ([GitHub](#))
- **Orbit** GitHub repo. ([GitHub](#))
- **Orbit** Uber engineering blog. ([Uber](#))
- **Odoo** Reordering Rules / Replenishment docs. ([Odoo](#))
- **ERPNext** Auto-creation of Material Request at Reorder level; Material Request docs. ([Frappe 文档](#))
- **XGBoost** GPU gpu_hist (tree methods) docs; legacy GPU notes. ([XGBoost 文档](#))
- **Ray** RLlib scaling guide; Ray actors. ([Ray](#))
- **OpenBoxes** Reorder Report & inventory levels; **OpenLMIS** stock-out reduction case. ([OpenBoxes 知识库](#))